

ECE-320 Lab 3: Modeling a Wheel System, Model Matching Control

Overview: In this lab we will utilize the dsPIC30F4011 to model a DC motor- wheel system. We will also begin to implement some realistic constraints on our controllers. One of the things you will discover is that you often need to be aware of limitations of both hardware and software when you try to implement a controller. We will then compare our modelled system with the real system for a few simple controllers.

You should start a new project, and copy all of the files from last weeks lab to the new project folder. This will keep you (hopefully) from screwing up anything that was already working.

PART A: Checking the scaling again for this wheel

You should be sure you have the same wheel system as last time, and check to see if you need to change the scaling. I would suggest running part G from the last lab and see if you get similar results. If not, you should determine the correct value of AD_scaling. Be careful of the scaling of the output!

For the remainder of this lab we will be making a model of the plant, including some of the nonlinearities of the DC motor. As you will see, this is still not an exact model. However, we will use this model in subsequent labs so try to do as well as you can. Be sure to record which wheel you are using since they are not all the same.

To get the parameters we need for modelling the plant, we need to be running the system open loop. Specifically, you need to do the following:

Set

```
reference_scaling = 1.0;
```

```
kp = 1.0;
```

In the PREFILTER section, we want a step input of amplitude 50, so comment out the part of the code that creates the step inputs (all of the *if* stuff), and set $R = 50.0$; as the last part of that section (no filtering, no reference scaling).

In the ERROR section, be sure the statement $error = R$; is **uncommented**, and the statement $error = R - Y$ is *commented out*.

Finally, we are going to want the control effort u written out (as an integer, not multiplied by 100). We want the value of u in the units of dutycycle (the variable dutycycle is really what we want). Modify your print statement (in the PRINT OUTPUT section) so four (integer) variables are printed out: the time ($\times 100$), the reference input ($\times 100$), the control effort, and the output ($\times 100$). *Do not modify this anywhere except near the print function at the end of the code!*

PART B: Modeling the plant as a first order system

A simple mathematical model of our plant in the time domain is

$$G_p(s) = \frac{K}{s + a}$$

Assuming a zero order hold (we will cover this later) we will get the discrete-time model

$$G_p(z) = \frac{\left(\frac{K}{a}\right)(1 - e^{-aT})}{z - e^{-aT}} = \frac{C}{z - c}$$

Note that we really only care about the discrete-time parameters C and c . If the input is then a step of amplitude R , we can write the output as

$$Y[n] = \frac{RC}{1 - c} [1 - c^n]u[n] = D[1 - c^n]u[n]$$

Since we know R and will determine c and D from the step response we can finally determine C as

$$C = \frac{D(1 - c)}{R}$$

Next you need to go to SecureCRT and prepare to log the step response. The MATLAB code we will use assumes the name of the file is **Step_Response_50**. Be sure to put this log file in a folder you can find (a good idea is in the current project folder.) Note that if you try this multiple times you need to toggle the **log session** choice. You toggle it once to start, and once to save the file. Each time you want a step response you should do this (otherwise it saves everything).

Now we are ready to measure the step response of the system. Once it starts let it run until steady state, but no longer than 15 seconds before you use your shutoff switch. *You should open **Step_Response_50** in MATLAB and edit it as necessary before any MATLAB programs try to read it. The C-code should output as integers: the time (scaled by 100) , the reference signal (scaled by 100) , the control effort (u), and the wheel speed (scaled by 100).*

Now run the MATLAB program **wheel_match.m** (available from the class Moodle site) and adjust the parameters D and c until you get a reasonably good fit for the data. Note that this program initially assumes a **delay** of two samples (which you may need to change.) It is most important to match the measured data for the first second or two and at the final value. Figure 1 shows the results for a typical wheel system. *Once you have a good overall fit you need to put a copy of your figure in your memo.*

Now change the value of R to 75.0 and try this again. Record the data in a different file this time (like **Step_Response_75**) and modify the MATLAB code to use the new file. You should use the same delay for this part that you did in the previous part. *Put a copy of this figure in your memo.*

In what follows use the average of the two values for c and C .

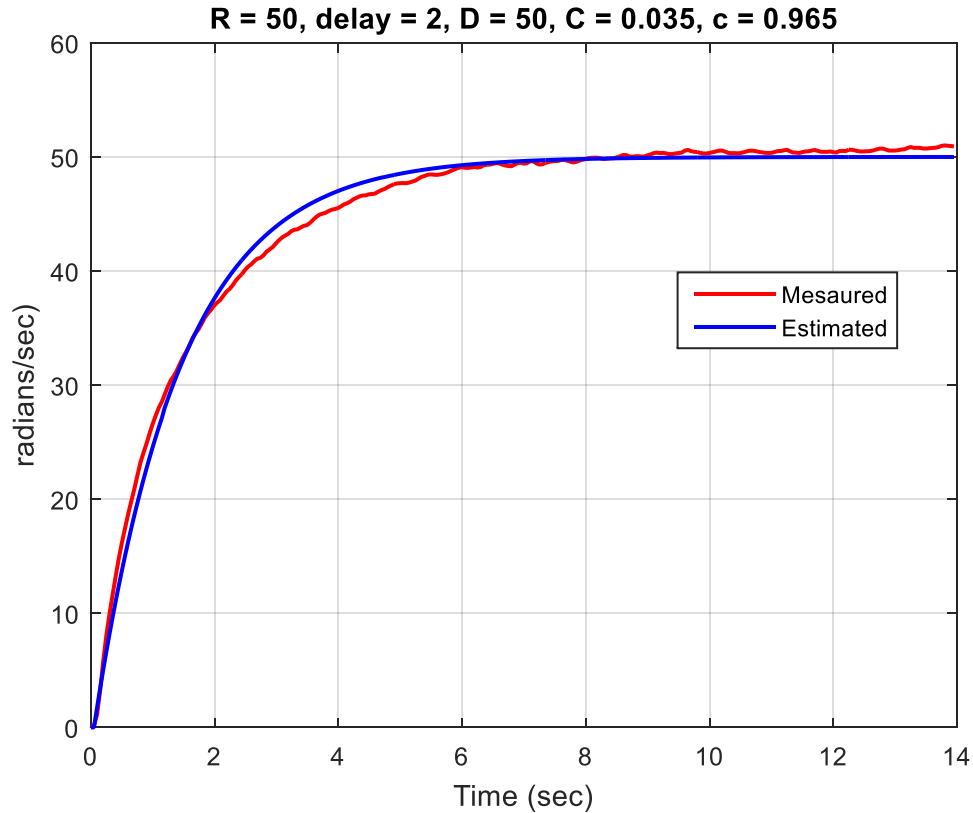


Figure 1. Typical results for modeling.

Part C: Including some nonlinearities in a Simulink model

Now we will start to make a Simulink model that includes some of the nonlinear effects of the system as shown in Figure 2. We will be modifying the values needed in the MATLAB/Simulink programs in the file **wheel_parameters.m** that you started last week. At this point we will enter some known values, you should modify the values of **AD_scale** to the value you are using in your C-code, and the **c**, **C**, and **delay** values should be what you just determined.

The file **DT_Openloop_driver.m** runs the Simulink file **DT_Openloop.slx**. As you can see, the Simulink file contains more than just the transfer function, it includes a few of the components we will use to model some of the nonlinearities of the system. At this point many of the parameters in the driver file are just dummies. The only nonlinear effect that has been included is a limitation on the control effort in the **Control Effort Limit** saturation block. *Note that all of the units in the Simulink model are in rad/sec.*

Once you have entered all the known values in **wheel_parameters.m**, run that code so all of the parameter values are in the MATLAB workspace.

The MATLAB script reads in the values from the file **Step_Response_50**, runs the Simulink model, and then compares the estimated and measured responses. As a check, do this now and compare the simulated output to your measured output. The results should look very similar to that you got with the **wheel_match.c** program. Note that this program also produces a plot of the control effort, which will be very useful to us later. For most of the graph, the control effort plots should be very close, or there is an error. Note that since we are filtering the system output, and not the input, there may be a slight delay between the measured signal and the output of the system.

Now we will set some of the other parameter values that model some of the system nonlinearity. Comment out the fixed value of R ($R=75.0$) and use the pot as the input (filtering it again). Compile and run the system and set the input to zero and stop the system. Slowly turn the pot all the way on, and see how fast the motor can go. Only let the motor run for at most 15-20 seconds, record the maximum speed of the motor, and then turn the pot back down to zero before shutting off the motor. Set the **Max_Speed** in **wheel_parameters.m** to this value (remember to divide by 100).

As you have probably noticed, the motor does not start spinning until the input gets to a certain value, and then it stops spinning at a different value. This is hysteresis and we will try and model it next. With the input at zero, slowly increase the value of the input until the motor (wheel) just starts to turn and record the value of the input (in rad/sec, be sure to divide by 100.) Now decrease the input signal slowly until the motor (wheel) stops and record this input (in rad/sec, be sure to divide by 100). Go through this process a few times until you feel you are getting consistent results (you might want to average the results for turning on and turning off). The **Relay_On** variable should be set to the value that makes the system start moving, and the **Relay_Off** variable should be set to the value where the system stops moving.

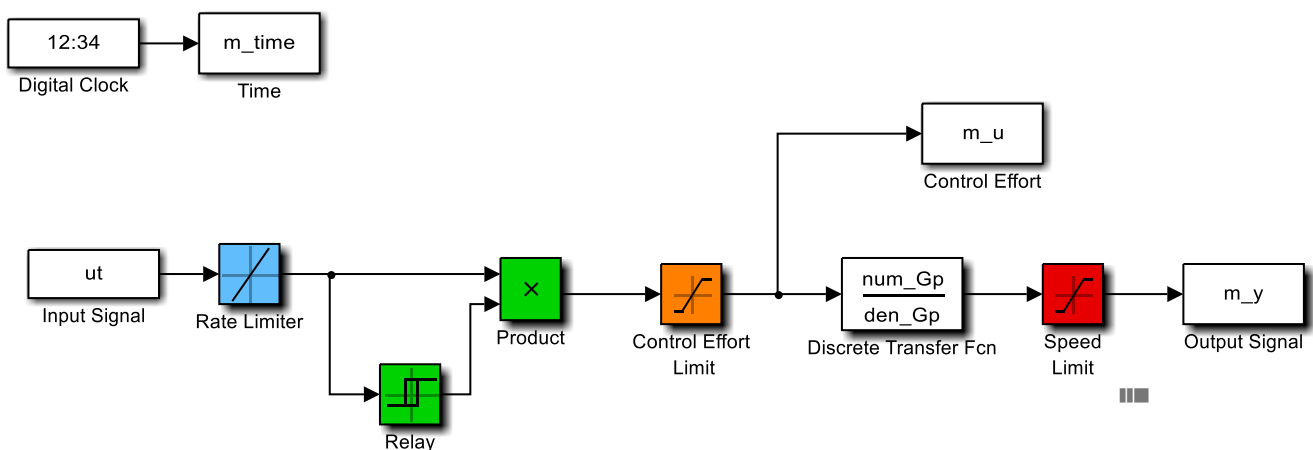


Figure 2. Simulink model of wheel including some nonlinearities.

PART D: Openloop control (again)

We now want to control our system using *openloop control*. You need to first run **wheel_parameters.m** to be sure all of the new values are in the MATLAB workspace. You will need to be sure the input pot is initially set to zero, and log a session with a new name (like **Openloop_Response**). Compile and run the system for about 30 seconds. As the system is running you should vary the input somewhat slowly, with a few plateaus. Rename the input file in **DT_Openloop_driver.m**, and run the program to compare the measured data with your model. *Remember that for a control system we want the output to match the input.* An example of a typical results is shown in Figure 3. Notice that since the motor only spins in one direction all we can do is coast to slow down. Also note that the control effort for the model and the real system are close but not exactly the same. *Include a copy of this figure in your memo. Your model and your measurements should be similar, but they will very likely be different in some ways.*

Part E : Limiting Slew Rate

The last thing we want to model is limiting the slew rate of the motor, or *how fast* we allow the control effort to increase. This will keep the motor from shredding the belt, and it is much more important in systems without such a powerful motor/amplifier. You will need to modify both your C- code and the **DT_Openloop_driver.m** file.

In the file **wheel_modelling_and_control.c** you need to:

- **Define** a constant at the top of the file **MAX_DELTA_U** (if it is not already defined) and set this value equal to 500.
- **Declare** a double in the main routine named **last_u** and initialize it to 0.0.;
- In the CONTROL EFFORT CONVERSION section of the code, modify the code so it looks like the following:

```
u = min(u, last_u+MAX_DELTA_U);  
u = min(u, MAX_DUTY);  
u = max(u,0.0);  
last_u = u;
```

Note that at this point in the program, the control effort u has been converted to a pwm duty cycle variable. Hence we are limiting size of the change we will allow in the overall dutycycle from one time step to the next.

In the file **wheel_parameters.m** you need to set the value of **MAX_DELTA_U** to match the C code and then re-run **wheel_parameters.m**.

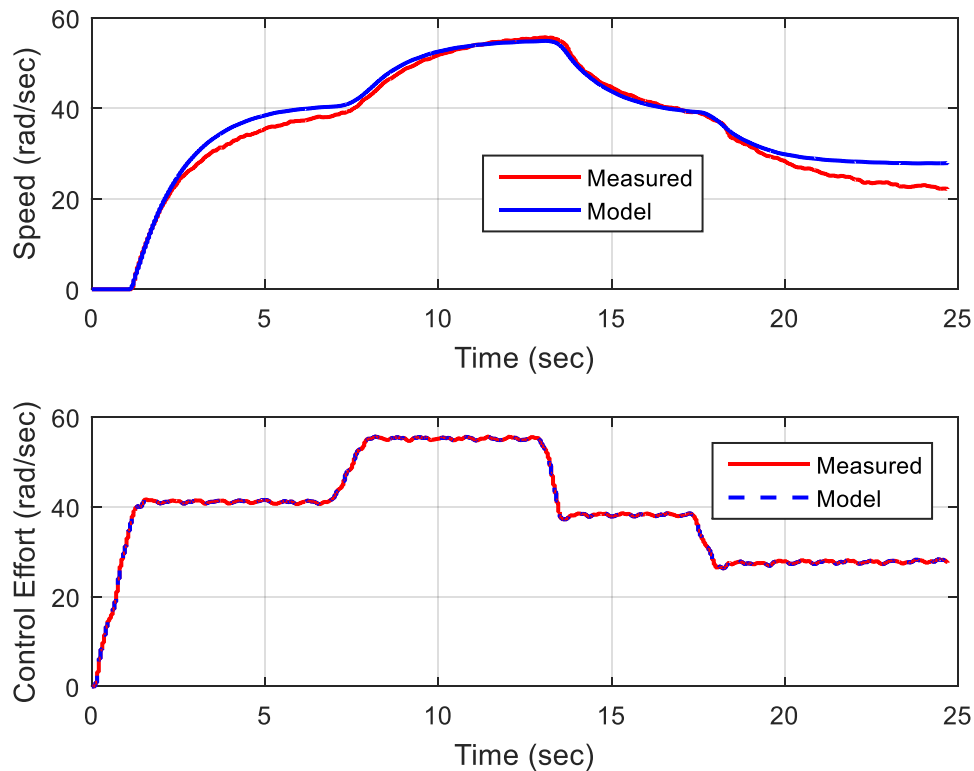


Figure 3. Openloop response comparing the model with the real system.

Rerun your system for a step input of 75 rad/second for around 10 seconds and run **DT_Openloop_driver.m** to compare the model with your real system. They should be fairly close (except for the control effort at the beginning.) Note at the beginning we are limiting the slope of the control effort, this is probably easiest to see in the log file. *Include your model/measured plot in your memo.*

PART F: Proportional control (again)

We now want to return to proportional control, a form of feedback control we used before. *In the script **wheel_parameters.m** and your C-code for the microcontroller set the value of **MAX_DELTA_U** to 1000 for the remainder of this lab, except, possibly, for the model matching part.*

Leave all of the other constants as they are.

In the PREFILTER section of the code, move code if necessary so the value of R is 75.0, followed by the scaling. Your code should be something like

```
R = 75.0;
```

```
R = R*reference_scaling;
```

and be sure **reference_scaling** is set to 1.0. In the ERROR section of the code, **uncomment** the code

```
error = R-Y;
```

and **comment out** the code

```
error = R;
```

In the CONTROLLER section of the code, you should have

```
u = kp*error;
```

and set **kp** = 5.0. Change the value of both **kp** and **reference_scaling** in your **wheel_parameters.m** file, to match this.

Finally, run **wheel_parameters.m** to put all of these variables in the MATLAB workspace.

Prepare to log the data measured (the MATLAB code assumes this file is called **Step_Response**, but feel free to change it.) Run the system until steady state (not more than 8 seconds though). Run the program **DT_PID_driver.m**, which at this point implements a proportional controller.

Now adjust your value of **reference_scaling** so the steady state value is between 74 and 76 rad/sec. Be sure to scale your output so the actual reference signal is output, not the scaled version (i.e., divide R by **reference_scaling** just before printing.) Be sure both your model and the C-code have the same values of **kp** and **reference_scaling**. Run the MATLAB code to compare your model with the real system. *Your measured and modeled results should be similar, but not identical. A typical (somewhat good) result is shown in Figure 4. Include your resultant graph in your memo.*

While the model is similar to the real system, it clearly lacks some information. However, there is a trade-off between how well a model fits the data and its usefulness. For us, result like that in Figure 4 are usually good enough to get us in the “ballpark” of the controller parameters we want. Then we usually just “tweak” them a bit.

Now change your code so the input is being read from the pot, then filtered, and then scaled by **reference_scaling**. Prepare to record your data and then run the system. Try to have an input with three or more plateaus (as shown in Figure 5), and then run the system using **DT_PID_driver.m**. A typical (somewhat good) example of the result is shown in Figure 5. *Include this figure in your memo.*

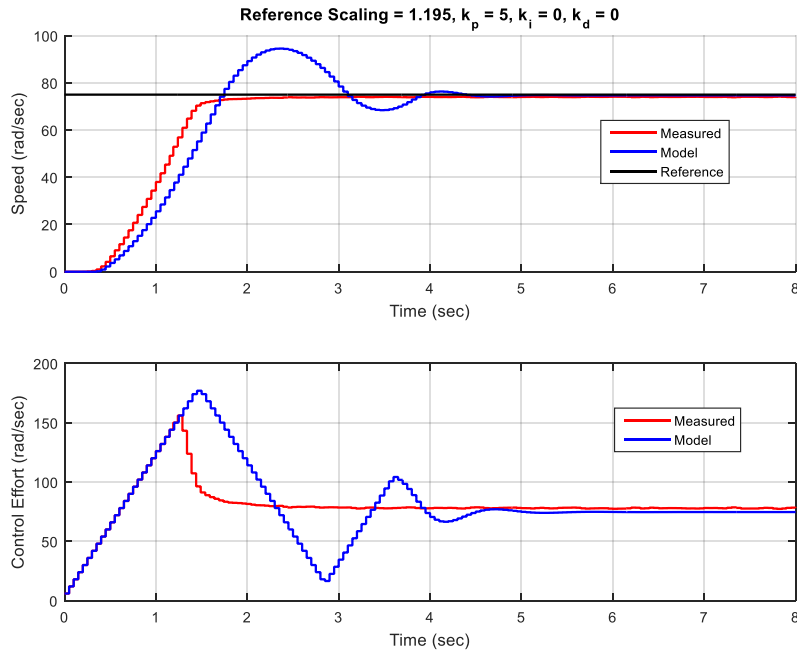


Figure 4. Comparison of real system with model using `DT_PID_driver.m`.

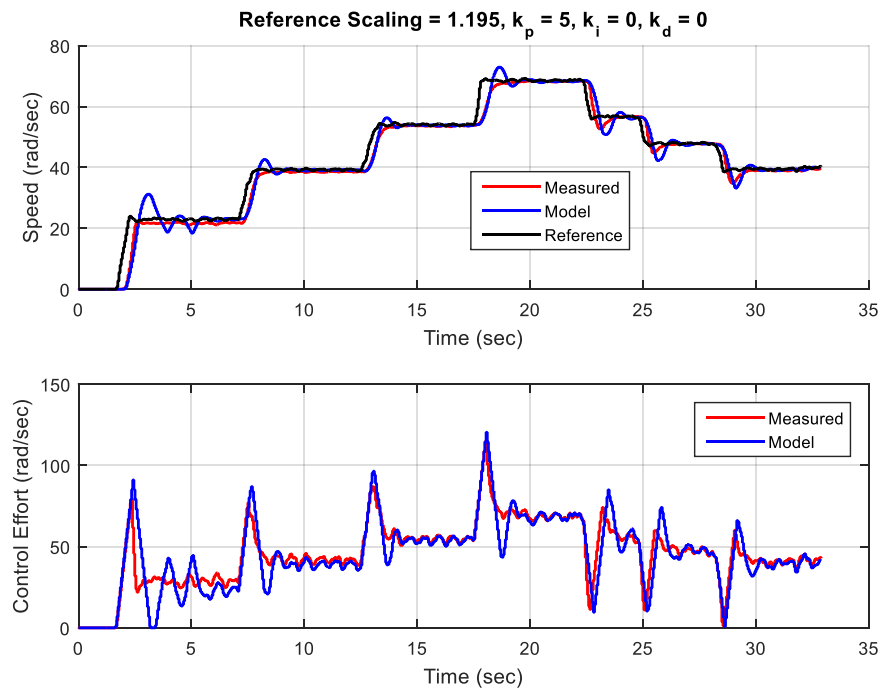


Figure 5. Model and real system with proportional controller.

PART G: Model Matching Control

For the last part of this lab, we will use Model Matching control. As you recall, for this type of control system we assume we know the plant, $G_p(z)$, we know the feedback transfer function, $H(z)$, and we know the desired closedloop transfer function, $G_o(z)$. However, while we do not know the controller, $G_c(z)$, we do know an equation relating all of these quantities:

$$G_o(z) = \frac{G_c(z)G_p(z)}{1 + H(z)G_c(z)G_p(z)}$$

Rearranging this equation we can solve for the controller as

$$G_c(z) = \frac{G_o(z)}{G_p(z)(1 - G_o(z)H(z))}$$

Note that in this equation we have a few things we have to be careful of:

- We need to be sure to eliminate pole-zero cancellations
- Our controller must be implementable, which means that the denominator must have an equal or higher power of z than the numerator.
- The controller must be stable, which means all of the poles must be inside the unit circle. However, we can have one pole at 1 (which will happen if we design everything correctly).

Most of what you need to do for this part is included in the file **DT_model_matching_driver.m**. This file assumes you have just run the previous part, so there is a file with a reference input and a time available. Also, it assumes you have run **wheel_parameters.m** so all of the necessary parameters are in the MATLAB workspace. Be sure **MAX_DELTA_U** is set equal to 1000.0, and **reference_scaling** is set equal to 1.0. Run **wheel_parameters.m** to be sure all of the parameters are correct. You may need to try and set **MAX_DELTA_U** to 10,000 to get good results.

The first thing you need to do (Step #1 in the code) is determine what you want the closed loop transfer function, $G_o(z)$, to be. Since your plant model is first order with no finite zeros, we will assume that is what we want our plant to look like. The code **d = poly([0.6 0]);** indicates that we want our closed loop transfer function to have one pole at 0.6, and one at zero. (A pole near zero represents faster response, while a pole at 1 represents slower response, which tends to be a smoother response. We will cover this later.) The poles at zero are for the delays we may need. You may have to add more poles at zero to make the system implementable (we will get to that later). The pole at 0.6 can be changed as you want, but it must be inside the unit circle. In order to get zero steady state error for a unit step, we want $G_o(1) = 1$. This is how we will determine the numerator of our desired transfer function, **alpha**. At this point you should run the code to see if the desired closed loop system responds as you would like (this code uses as input the data file **Step_Response** from the last part as the input.)

Once you have what you think is a good desired closed loop transfer function, you need to **uncomment** the next section of code (Step #2). In this section of the code, we enter the model of the plant we have, including the delays we may have estimated (we include the delays in the transfer function D, *so modify this transfer function to match the delays in your system*). Next, we determine the controller based on all of the information we have. The command **minreal** takes care of pole/zero cancellations (MATLAB sets the tolerance, but you can change it if you want). The next piece of code removes all of the coefficients in $G_c(z)$ that are smaller than the value **tol**. This is to eliminate coefficients that really should be zero. Run the code again, and you should get two plots, the second one should look like the first one, it is basically a check. However, if you did not get a second graph, most likely MATLAB is complaining that you have a transfer function for the controller that is not implementable. You can tell the transfer function is not implementable if the highest power of z in the numerator is larger than the highest power of z in the denominator. If that happens, go back to Step#1, and add another pole at zero in the desired closed loop response. Repeat this until the code runs without complaining.

Next, uncomment the section for Step#3. In this part, we are including all the nonlinearities that our control system has, including the maximum slew rate and maximum control effort. At this point you may want to change the desired closed loop response to get a better response for the system with all of the constraints. If you get a response but it does not look like your system is tracking the input well, try changing the value of **MAX_DELTA_U** in **wheel_parameters.m**, run that script again, and then run **DT_model_matching_driver.m** again. Be sure your control effort is not too large (see the plots).

However, if you are satisfied with your response, we can go to implement our controller based on model matching. In your C-code, be sure **reference_scaling** is set to 1.0, and **MAX_DELTA_U** is set to the same thing as it is in **wheel_parameters**.

Now we need to implement the controller. This should be pretty simple, since we really already know how to do this. Recall the controller is just a transfer function, as is a filter. (A controller is just a specific kind of filter.) The program **DT_model_matching_driver.m** should have written to your screen (among many other things) two arrays: **Bc** and **Ac**. These contain the “filter” coefficients you will need. You will have to make two new arrays in your C-code initialized with these values, and two other arrays for the input error values (**error_in**) and the control effort (**u_out**). When you are done the control effort should still be called **u**, since that is what the rest of the program expects. Be careful with the size of the arrays (you will probably need to initialize a variable **Nu** and set it equal to the size of the arrays.) Now you pretty much need to copy, paste, and rename from some other part of the code where you filtered. **However, do not include any checks on the size or sign of u**, since this is done elsewhere in the program. This section of code should be in the CONTROLLER part of the code. There should be no other assignments to the control effort (you may need to comment the old ones out.)

Run your system and log the results to an appropriately named file. Try to include a number of plateaus in your input. Modify **DT_model_matching_driver.m** to read this file, and then uncomment Part 4, which should plot the results from the model and the real system. While we want the model to match the system reasonably well, it is more important that the real systems tracks the input. It is also very

common for the real system and model to match better in terms of the wheel speed than the control effort. A typical response is shown in Figure 6. If your system does not follow the reference input very well, you may need to change the poles of the desired closed loop system and create another controller. *Include your final figure in your memo.*

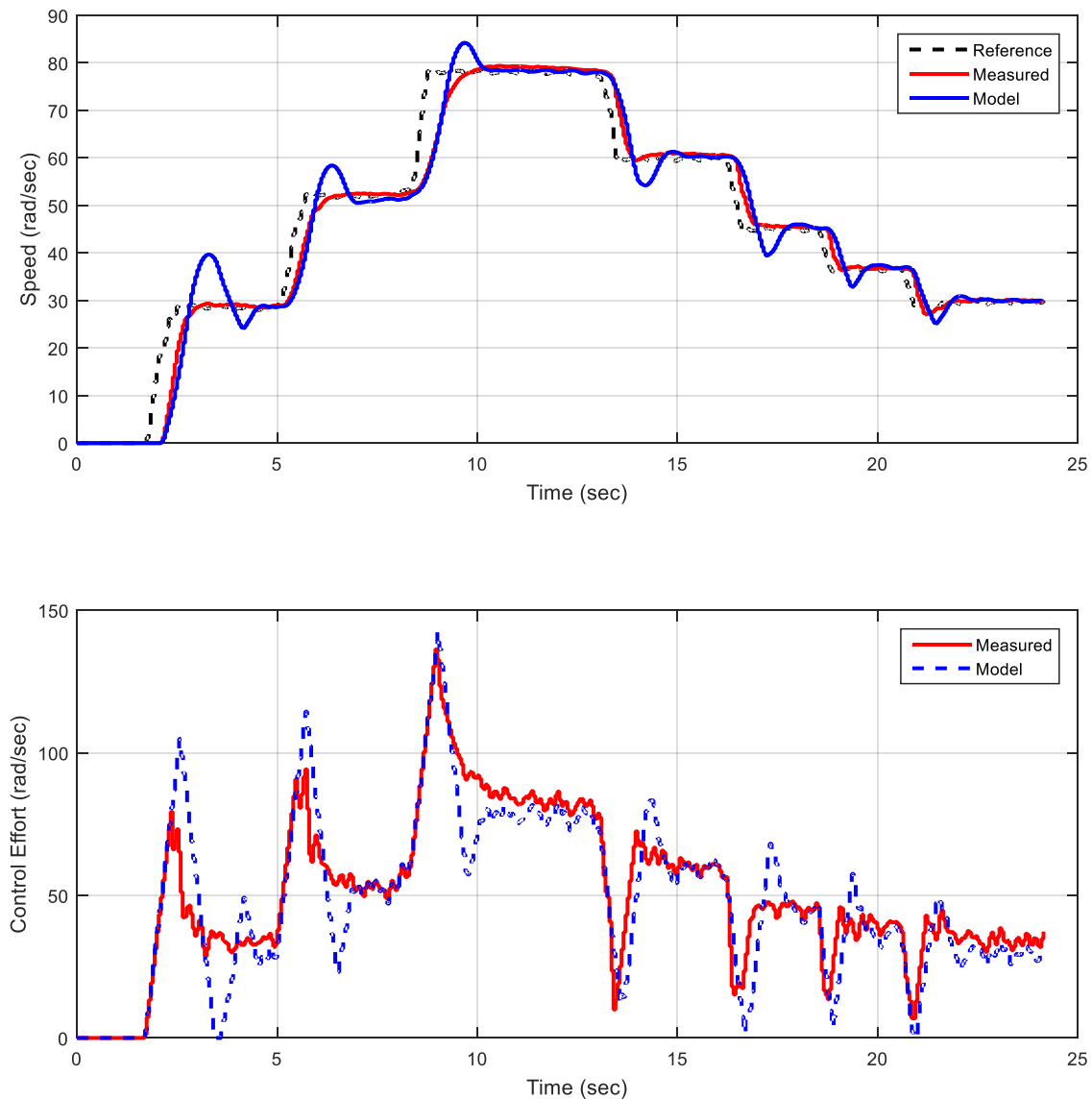


Figure 6. Model matching results, comparing the real and simulated systems using model matching.

